

Computational Pipeline Summary: Zoobenthic Community Index Construction

Methods, Data Flow, and Quality Assessment

March 5, 2026

Contents

1	Overview	2
2	Data Dictionary and Notation	2
2.1	Workbook Structure	2
2.2	Key Symbols and Variable Types	3
2.3	Characters of the Raw Data	3
2.3.1	Taxa Data in Octave Scale	3
2.3.2	Mean Particle Size — MPS(Phi)	4
2.3.3	Selection of 16 Chemical Variables	4
3	Core Analytical Stream	4
3.1	Stage 1 — Pollution PCA	4
3.2	Stage 2 — Taxa Assemblage Clustering	5
3.3	Hindsight Relabel and ANOVA	6
3.4	Stage 3 — LDA Habitat Classification	6
3.5	Stage 4 — Bray–Curtis NMDS and ZCI Construction	7
3.6	Stage 5 — Piecewise Quantile Regression	9
4	Complementary / Validation Methods	10
4.1	Stage RDA — Redundancy Analysis	10
4.2	Role of ANOVA in the Framework	11
5	Artifact and Intermediate-Value Map	11
6	Quality Assessment and Potential Issues	13
6.1	ANOVA Assumption Checking	13
6.2	Bray–Curtis and NMDS: Where Each Operates	13
6.3	LDA Cross-Validation Leakage	14
7	TITAN Embedding Plan	14
7.1	Background	14
7.2	TITAN Algorithm Summary	15
7.3	Integration Points in the Current Framework	15
8	Summary of Recommendations	16

1 Overview

The pipeline constructs a *Zoobenthic Community Index* (ZCI) that quantifies benthic community condition along a sediment-contamination gradient in the St. Clair–Detroit River System (SCDRS) corridor. The corridor-wide workflow is orchestrated by `run_SCDRS_full_pipeline.py` and proceeds through the following stages:

1. **Stage 1** — Pollution PCA: derive a composite *Pollution Score* from 16 sediment-chemistry variables.
2. **Stage RDA** — Redundancy Analysis (complementary validation): quantify how much of the community variation is explained by environmental predictors.
3. **Stage 2** — Taxa assemblage clustering: partition least-polluted (reference) sites into habitat groups using Ward’s hierarchical clustering on taxa data.
4. **Hindsight relabel + ANOVA**: merge small clusters and test environmental/taxa differences across habitat groups.
5. **Stage 3** — LDA classification: train a Linear Discriminant Analysis on reference sites to classify all remaining (non-reference) sites into habitat groups.
6. **Stage 4** — Bray–Curtis NMDS + ZCI: ordinate taxa communities by non-metric multi-dimensional scaling (NMDS) on Bray–Curtis dissimilarities, then compute a ZCI for every site.
7. **Stage 5** — Piecewise quantile regression: model the ZCI–Pollution relationship with piecewise linear quantile regression and wild-bootstrap confidence intervals.

Stages 1–4 and the hindsight step form the **core analytical stream**; Stage RDA is **complementary/validation**; Stage 5 is a **post-hoc inference** layer. A prospective **TITAN** (Threshold Indicator Taxa ANalysis) module is discussed in section 7.

Each stage reads one or more *artifact* Excel workbooks produced by its predecessors, adds new columns, and writes an updated artifact. All workbooks use a three-level **MultiIndex** column header (`block`, `level`, `variable`).

2 Data Dictionary and Notation

2.1 Workbook Structure

The master workbook (`complete_env_taxa_chemical_Feb_3.xlsx`) is read by `read_study_data()` (module `zci.io.readers`) into a `pandas.DataFrame` with a three-level column **MultiIndex**:

$$\left(\underbrace{\text{block}}_{\text{e.g. "chemical"}}, \underbrace{\text{level}}_{\text{"raw"}}, \underbrace{\text{variable}}_{\text{e.g. "Cu"}} \right).$$

The helper `extract_block(data, block, level)` returns a flat `DataFrame` containing only the columns belonging to the requested block and level. Four blocks are used:

Block key	Content
<code>chemical / raw</code>	16 sediment-chemistry concentrations
<code>taxa / raw</code>	16 benthic invertebrate taxa in octave scale
<code>environmental / raw</code>	Physical habitat variables (depth, DO, temperature, MPS, LOI, velocity)
<code>sample_info / raw</code>	Site identifiers, coordinates, waterbody

2.2 Key Symbols and Variable Types

Throughout, we index n sites by i and suppress the site subscript where unambiguous.

Symbol	Dimensions	Code name	Description
C	$\mathbb{R}^{n \times 16}$	<code>pollution_raw</code>	Raw sediment-chemistry matrix (sites $\times$ 16 variables)
$\tilde{\mathbf{C}}$	$\mathbb{R}^{n \times 16}$	<code>pollution_transformed</code>	$\log_2$ -transformed chemistry: $\tilde{c}_{ij} = \log_2(1 + c_{ij})$
O	$\mathbb{R}^{n \times p}$	<code>taxa_all</code>	Taxa matrix in <i>octave</i> scale ($p = 16$ study taxa)
P	$\mathbb{R}^{n \times p}$	<code>taxa_relabd</code>	Relative-abundance matrix recovered from octave; rows sum to 1
E	$\mathbb{R}^{n \times q}$	<code>env_ref</code> or <code>env_data</code>	Environmental predictor matrix ($q = 5$ unless otherwise stated)
s	$\mathbb{R}^n$	<code>Pollution_Score</code>	Composite pollution score (Stage 1 output)
g	$\{1, \dots, K\}^n$	<code>Predicted_Cluster</code>	Habitat-cluster label for each site (Stage 3 output)
x	$\mathbb{R}^{n \times 2}$	<code>coords_df [NMDS1, NMDS2]</code>	NMDS ordination coordinates (Stage 4)
z	$[0, 1]^n$	<code>ZCI</code>	Zoobenthic Community Index (Stage 4 output)

2.3 Characters of the Raw Data

2.3.1 Taxa Data in Octave Scale

The 16 taxa variables are stored in **octave-transformed** form. The octave scale encodes the original proportional abundance p_{ij} (taxon j at site i) as

$$o_{ij} = \log_2(100(p_{ij} + 0.01)). \quad (1)$$

Hence o_{ij} increases monotonically with p_{ij} but is not a count; absolute abundances are **not recoverable** from the data.

The inverse used by the pipeline (`octave_to_relative_abundance()` in module `zci.core.transforms`) is

$$\hat{p}_{ij} = \max(0, 2^{o_{ij}} - 0.0625), \quad p_{ij}^{\text{RA}} = \frac{\hat{p}_{ij}}{\sum_{j'} \hat{p}_{ij'}}, \quad (2)$$

producing a row-normalised relative-abundance matrix **P** ($p_i^{\text{RA}} \in [0, 1]$, $\sum_j p_{ij}^{\text{RA}} = 1$).

Usage in downstream stages.

- **Stage 2 clustering** uses octave values directly (identity transform) when `taxa_transform="octave"` is specified.
- **Stage 4 NMDS / ZCI** converts to relative abundance via (2) so that the Bray–Curtis dissimilarity operates on compositional proportions.
- **ANOVA** (hindsight step) presents per-taxon means on the un-transformed octave scale for interpretability.
- **RDA** can receive either octave or Hellinger-transformed taxa (configurable via `taxa_transform`).

2.3.2 Mean Particle Size — MPS(Phi)

Sediment grain size is stored as **MPS (Phi)**, the mean particle size after the Krumbein phi transformation:

$$\phi = -\log_2(\text{particle size in mm}). \quad (3)$$

The column **MPS (Phi)** exists in the input workbook as a pre-computed field; the pipeline reads it directly and does not recompute ϕ from raw particle-size measurements. It enters Stage RDA and Stage 3 (LDA) as one of the environmental predictors **E**.

2.3.3 Selection of 16 Chemical Variables

The 2004/5 corridor-wide survey measured 19 elements (18 metals plus arsenic); the 1991 and 1999 Detroit River surveys reported 13 and 19 elements respectively. The intersection yielded **13 elements** (12 metals plus arsenic) that were common to all three surveys.

All three surveys reported PCB congener concentrations, but methodology and detection limits changed substantially between 1990 and 2004, making congener-by-congener analysis unreliable. Therefore the **sum of PCBs** (Σ PCBs, recorded as **total PCB**) was used instead.

Pesticide and petrochemical data were variably available across surveys. Two single-variable surrogates were chosen:

- **p,p'-DDE**: a representative insecticide degradation product (surrogate for agricultural pesticides).
- **OCS** (octachlorostyrene): a petrochemical byproduct (surrogate for industrial petrochemical contamination).

This yields a total of $13 + 1 + 1 + 1 = 16$ chemical variables compiled into the site-by-contaminant matrix **C**. The exact column names used in the code (constant `POLLUTION_VARS_2008` in module `zci.pipeline.pollution_assessment`) are:

Co, Al, Ni, Mn, Fe, Cr, Cu, Hg, Pb, Zn, total PCB, Cd, OCS, p,p'-DDE, As, Ca.

3 Core Analytical Stream

3.1 Stage 1 — Pollution PCA

Module. `zci.pipeline.pollution_assessment.pollution_pca_pipeline()`.

Input. Chemical block $\mathbf{C} \in \mathbb{R}^{n \times 16}$ (`extract_block(data, "chemical", "raw")[POLLUTION_VARS_2008]`).

Step 1 — Log transformation. Each concentration is log-transformed element-wise (`zci.core.transforms`).

$$\tilde{c}_{ij} = \log_2(1 + c_{ij}), \quad i = 1, \dots, n, \quad j = 1, \dots, 16. \quad (4)$$

This yields the transformed matrix $\tilde{\mathbf{C}} \in \mathbb{R}^{n \times 16}$ (`pollution_transformed: pd.DataFrame`).

Step 2 — PCA. Principal Component Analysis is performed on $\tilde{\mathbf{C}}$ (`zci.core.pca.run_pca`):

1. Centre: $\mathbf{T} = \tilde{\mathbf{C}} - \mathbf{1} \tilde{\mathbf{c}}^\top$.
2. SVD: $\mathbf{T} \approx \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$ with k retained components (default $k = 5$).
3. **Scaled loadings:** $L_{jm} = v_{jm} \sqrt{\lambda_m}$, where λ_m is the m -th eigenvalue and v_{jm} the corresponding eigenvector element. (`result.loadings: pd.DataFrame`, shape $16 \times k$.)
4. **Site scores:** $\mathbf{S}_{\text{raw}} = \mathbf{T} \mathbf{V}_k \in \mathbb{R}^{n \times k}$ (`result.scores_raw`).

5. **Standardised scores:** each column of $\mathbf{S}_{\text{raw}}$ is min–max rescaled to $[0, 1]$ (`result.scores: pd.DataFrame`):

$$s_{im}^* = \frac{s_{im} - \min_i s_{im}}{\max_i s_{im} - \min_i s_{im}}. \quad (5)$$

Note. The PCA is fit on $\tilde{\mathbf{C}}$ *without* prior unit-variance scaling; variables with larger log-scale variance therefore have more influence on the leading PCs. This is a deliberate design choice, not an oversight, but it means that the pollution score is weighted towards high-variance contaminants.

Step 3 — Composite Pollution Score. Selected PCs (by default all k) are individually min–max-scaled and then summed with equal weights (`zci.core.transforms.composite_pollution_score`):

$$s_i = \sum_{m=1}^k w_m \cdot s_{im}^*, \quad w_m = 1 \quad \forall m \text{ (default)}. \quad (6)$$

Result: `Pollution_Score: pd.Series` ($\mathbf{s} \in \mathbb{R}^n$). Higher values indicate greater contamination.

Artifact written. `01_updated_data.xlsx` — the original workbook augmented with columns PC1–PC k and `Pollution_Score`, under the block header ("`01_pollution_assessment`", "`raw`", *).

3.2 Stage 2 — Taxa Assemblage Clustering

Module. `zci.pipeline.taxa_assemblage.taxa_assemblage_pipeline()`.

Input.

- Taxa block $\mathbf{O} \in \mathbb{R}^{n \times 16}$ (octave scale).
- Pollution Score $\mathbf{s}$ from Stage 1 artifact.

Step 1 — Reference-site selection. A Boolean mask selects the least-polluted fraction of sites (`zci.core.clustering.select_reference_sites`):

$$\mathcal{R} = \{i : s_i \leq s_{(n_{\text{ref}})}\}, \quad n_{\text{ref}} = \lfloor 0.20 n \rfloor, \quad (7)$$

where $s_{(n_{\text{ref}})}$ is the n_{ref} -th smallest value of $\mathbf{s}$. Default: bottom 20 % of pollution scores. Result: `ref_mask: pd.Series[bool]`.

Step 2 — Taxa transform. When `taxa_transform="octave"` (the default), the taxa matrix is used as-is; no numerical transformation is applied. The function `zci.core.transforms.octave_transform()` simply returns a copy.

Step 3 — Ward’s hierarchical clustering. `zci.core.clustering.ward_cluster(taxa_ref, n_clusters=K)` performs agglomerative clustering with Ward’s linkage (minimum within-cluster variance) and Euclidean distance:

$$d_{\text{merge}}(A, B) = \sqrt{\frac{2|A||B|}{|A| + |B|}} \|\bar{\mathbf{o}}_A - \bar{\mathbf{o}}_B\|, \quad (8)$$

where $\bar{\mathbf{o}}_A$ denotes the centroid of cluster A in octave space. The dendrogram is cut at K clusters (default $K = 3$ for the corridor-wide run, later merged to $K = 2$ in the hindsight step).

Result: `labels_ref: pd.Series` (1-indexed cluster labels for reference sites only; non-reference sites are NaN).

Artifact written. `02_updated_data.xlsx` — block ("02_taxa_assemblage", "raw", "Cluster") populated for $|\mathcal{R}|$ reference sites.

3.3 Hindsight Relabel and ANOVA

Module. `run_hindsight_relabel.py`; ANOVA core in `zci.core.anova.anova_table()`.

Cluster relabelling. After inspecting Stage 2 dendrograms and cluster membership, smaller clusters may be merged. In the corridor-wide run the mapping $\{3 \rightarrow 1\}$ is applied, effectively merging the smallest cluster into Cluster 1, yielding $K = 2$ habitat groups.

One-way ANOVA. For each environmental variable e (from $\mathbf{E}$) and each taxon (from $\mathbf{O}$, restricted to reference sites), a one-way ANOVA across the K clusters is computed:

$$F = \frac{SS_{\text{between}}/(K-1)}{SS_{\text{within}}/(n_{\text{ref}}-K)}, \quad (9)$$

where

$$SS_{\text{between}} = \sum_{g=1}^K n_g (\bar{e}_g - \bar{e})^2, \quad SS_{\text{within}} = \sum_{g=1}^K \sum_{i \in g} (e_i - \bar{e}_g)^2.$$

The p -value is obtained from `scipy.stats.f_oneway`.

The code supports an optional pre-ANOVA transform controlled by the parameter **transform**:

- "none" — no transformation (default for the corridor run).
- "log" — $\ln(1+x)$ applied to each variable.
- "boxcox" — Box-Cox power transformation (with fallback to $\ln(1+x)$ on failure).

Per-cluster means $\pm$ SEM are reported on the *untransformed* scale regardless of the ANOVA transform used.

Outputs.

- `anova_env.xlsx`, `anova_taxa.xlsx` — ANOVA summary tables with F , p , cluster means $\pm$ SEM.
- `cluster_panel.png` — map + environmental bars + taxa bars.
- `02_hindsight_updated_data.xlsx` — relabelled artifact (block column **Cluster** updated).

3.4 Stage 3 — LDA Habitat Classification

Module. `zci.pipeline.lda_classification.lda_pipeline()`; core in `zci.core.lda`.

Input.

- Environmental matrix $\mathbf{E}_{\mathcal{R}}$ for reference sites, comprising the $q = 5$ variables: Measured Depth (m), Water DO Bottom (mg/L), Temperature (oC), MPS (Phi), LOI (%).
- Cluster labels $\mathbf{g}_{\mathcal{R}}$ from the hindsight artifact.
- Pollution Score $\mathbf{s}$ (for reference-site identification).

Step 1 — Standardisation. When `standardize=True` (default), each environmental variable is z -scored using `sklearn.preprocessing.StandardScaler`:

$$e_{ij}^* = \frac{e_{ij} - \bar{e}_j}{\sigma_j}, \quad (10)$$

fitted *on the reference sites only*. The same scaler object is stored in the `LDAFit` container and reapplied to non-reference sites at prediction time (`zci.core.lda.predict_sites`).

Step 2 — LDA fitting. `sklearn.discriminant_analysis.LinearDiscriminantAnalysis` is fit on $(\mathbf{E}_{\mathcal{R}}^*, \mathbf{g}_{\mathcal{R}})$. For K classes and q predictors the model extracts $\min(K - 1, q)$ linear discriminant axes, each a linear combination of the standardised environmental variables that maximises the between-class to within-class variance ratio:

$$\text{LD}_m = \mathbf{w}_m^\top \mathbf{e}^*, \quad \mathbf{w}_m = \arg \max_{\mathbf{w}} \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}, \quad (11)$$

where $\mathbf{S}_B$ and $\mathbf{S}_W$ are the between- and within-class scatter matrices respectively.

Step 3 — Variable importance (Wilks' Lambda). `zci.core.lda.wilks_lambda_importance()` performs a drop-one analysis. For each variable j :

$$\Lambda_{\text{full}} = \frac{|\mathbf{W}|}{|\mathbf{T}|}, \quad (12)$$

$$\Lambda_{-j} = \text{Wilks' } \Lambda \text{ with variable } j \text{ removed}, \quad (13)$$

$$\Delta\Lambda_j = \Lambda_{-j} - \Lambda_{\text{full}}, \quad (14)$$

and an approximate F -statistic is computed:

$$F_j = \frac{\Delta\Lambda_j / \Lambda_{\text{full}}}{(K - 1)^{-1} (n_{\text{ref}} - K - q + 1)^{-1}}. \quad (15)$$

Overall model significance is assessed via Bartlett's χ^2 approximation:

$$\chi^2 = -\left(n_{\text{ref}} - 1 - \frac{q + K}{2}\right) \ln \Lambda_{\text{full}}, \quad \text{df} = q(K - 1). \quad (16)$$

Step 4 — Monte Carlo cross-validation (MCCV). `zci.core.lda.monte_carlo_cv()` repeats stratified 80/20 train/test splits for N_{cv} iterations (default 1000):

1. *Globally* standardise (z-score) the reference environmental data.
2. For each iteration: draw a stratified 80 % training set, fit LDA, predict on the held-out 20 %, record accuracy.
3. Report mean $\pm$ std accuracy, per-class precision / recall / F1, and an aggregated confusion matrix.

Caveat: the z-score standardisation in the current code is applied *globally before* the CV splits, not refitted inside each fold. This is a minor data-leakage issue; reported CV accuracies may be slightly optimistic.

Step 5 — Classification of non-reference sites. `zci.core.lda.predict_sites()` applies the fitted LDA (with the stored scaler) to the environmental data of all non-reference sites, producing:

- **Predicted_Cluster:** `pd.Series` — the habitat label g_i for each non-reference site.
- **probabilities:** `pd.DataFrame` — posterior class probabilities.
- **An_Is_Reference:** `pd.Series[bool]` flag.

Artifact written. `03_updated_data.xlsx` — `block("03_lda_classification", "raw", *)`, adding columns `Predicted_Cluster`, `Is_Reference`, per-class probabilities, and LD coordinates.

3.5 Stage 4 — Bray–Curtis NMDS and ZCI Construction

Module. `zci.pipeline.bray_curtis_nmds.nmds_pipeline()`; core in `zci.core.nmds` and `zci.core.zci_scores`.

Input.

- Taxa block **O** (octave scale) from the original workbook.
- Pollution Score **s** from Stage 1 artifact.
- Predicted Cluster **g** and **Is_Reference** from Stage 3 artifact.

Step 1 — Octave to relative abundance. The taxa matrix is converted to relative abundance via `octave_to_relative_abundance()` as given in (2), producing $\mathbf{P} \in \mathbb{R}^{n \times p}$ (`taxa_relabd: pd.DataFrame`).

Step 2 — Endpoint construction (per cluster). For each cluster g , the n_{ep} sites with the lowest Pollution Score form the **reference endpoint** and the n_{ep} highest form the **degraded endpoint** (`zci.core.nmds.build_endpoints`):

$$\bar{\mathbf{p}}_{\text{ref}} = \frac{1}{n_{\text{ep}}} \sum_{i \in \mathcal{E}_{\text{ref}}} \mathbf{p}_i^{\text{RA}}, \quad \bar{\mathbf{p}}_{\text{deg}} = \frac{1}{n_{\text{ep}}} \sum_{i \in \mathcal{E}_{\text{deg}}} \mathbf{p}_i^{\text{RA}}, \quad (17)$$

where $\mathcal{E}_{\text{ref}}$ and $\mathcal{E}_{\text{deg}}$ are the index sets of the n_{ep} extreme sites. These two “synthetic sites” are appended to the cluster’s taxa matrix.

Step 3 — Bray–Curtis dissimilarity. The pairwise Bray–Curtis dissimilarity between every pair of sites (including endpoints) in the augmented relative-abundance matrix is computed (`scipy.spatial.distance.pdist(..., metric="braycurtis")`):

$$d_{\text{BC}}(i, i') = \frac{\sum_{j=1}^p |p_{ij}^{\text{RA}} - p_{i'j}^{\text{RA}}|}{\sum_{j=1}^p (p_{ij}^{\text{RA}} + p_{i'j}^{\text{RA}})}, \quad (18)$$

yielding $\mathbf{D} \in [0, 1]^{(n_g+2) \times (n_g+2)}$, where n_g is the number of real sites in cluster g .

This is the first place where Bray–Curtis is computed; it feeds the NMDS ordination.

Step 4 — Iterative non-metric MDS. `zci.core.nmds.iterative_nmds()` embeds $\mathbf{D}$ into $\mathbb{R}^2$ via non-metric multidimensional scaling (NMDS):

1. **Pass 1:** `sklearn.manifold.MDS` with `metric=False` (non-metric) and `n_init=10` random starts.
2. **Passes 2–3:** re-initialise MDS with the previous solution; `n_init=4` additional random starts per refinement.
3. Total: 3 iterations of SMACOF (Scaling by MAjorising a Complicated Function).

The algorithm minimises the *stress* criterion:

$$\text{Stress} = \sqrt{\frac{\sum_{i < i'} (f(d_{\text{BC}}(i, i')) - \|\mathbf{x}_i - \mathbf{x}_{i'}\|)^2}{\sum_{i < i'} \|\mathbf{x}_i - \mathbf{x}_{i'}\|^2}}, \quad (19)$$

where f is a monotone regression function (the “non-metric” part).

Step 5 — PCA rotation and axis flip. `zci.core.nmds.pca_rotate()` rotates the NMDS coordinates so NMDS1 captures the maximum variance. Then `zci.core.nmds.flip_axis()` checks the Pearson correlation r between NMDS1 and **s** (Pollution Score) for the real sites:

$$\text{if } r(\text{NMDS1}, \mathbf{s}) > 0 \implies \text{NMDS1} \leftarrow -\text{NMDS1}. \quad (20)$$

This enforces a convention where NMDS1 is *negatively* correlated with pollution (cleaner sites on the positive end).

Interpretive caveat: this axis-flip is enforced by design, so the resulting negative NMDS1–Pollution correlation is partly an artefact of the orientation convention and should not be interpreted as a discovered ecological relationship.

Step 6 — Weighted-average species scores. `zci.core.nmds.weighted_average_scores()` assigns each taxon a position in NMDS space:

$$\text{WA}_j = \frac{\sum_{i=1}^{n_g} p_{ij}^{\text{RA}} \mathbf{x}_i}{\sum_{i=1}^{n_g} p_{ij}^{\text{RA}}}. \quad (21)$$

Step 7 — ZCI computation. The pipeline currently uses the **BC-Direct** method (`zci.core.zci_scores.zci`) as the default ZCI:

$$z_i = \frac{d_{\text{BC}}(i, \mathcal{E}_{\text{deg}})}{d_{\text{BC}}(i, \mathcal{E}_{\text{ref}}) + d_{\text{BC}}(i, \mathcal{E}_{\text{deg}})}, \quad (22)$$

where $d_{\text{BC}}(i, \mathcal{E})$ is the Bray–Curtis distance from site i ’s relative-abundance vector to the centroid of the endpoint set $\mathcal{E}$ (computed via `scipy.spatial.distance.cdist`).

*This is the second place where Bray–Curtis is applied; it operates directly on relative-abundance vectors and does **not** use the NMDS coordinates.* Values near 1 indicate a community similar to the reference endpoint; values near 0 indicate a degraded community.

The default per-cluster endpoint sizes in the corridor run are:

Cluster	n_{ep} (ZCI)	Method
1	15	BC-Direct
2	3	BC-Direct

ZCI–Pollution correlations (Pearson and Spearman) are computed via `zci.core.zci_scores.zci_correlate` and saved in `zci_summary.xlsx`.

Alternative ZCI methods (available but not used by default). The module `zci.core.zci_scores` also implements:

- **Distance:** $z_i = d_{\text{deg}} / (d_{\text{ref}} + d_{\text{deg}})$ using Euclidean distances in NMDS space.
- **Projection:** project sites onto the directed DEG→REF line in NMDS space, normalised to $[0, 1]$.
- **Centroid-Proj:** project onto a centroid-defined axis using real-site pollution extremes (rather than synthetic endpoints).

Artifact written. `04_updated_data.xlsx` — block ("04_bray_curtis_nmds", "raw", *), adding columns NMDS1, NMDS2, ZCI, ZCI_Method.

3.6 Stage 5 — Piecewise Quantile Regression

Module. `zci.pipeline.piecewise_qr_pipeline.pqr_pipeline()`; core in `zci.core.piecewise_qr`.

Input.

- **s** (Pollution Score) from Stage 1.
- **g** (Predicted Cluster) from Stage 3.
- **z** (ZCI) from Stage 4.

Model. For each cluster and a grid of quantile levels $\tau \in \{0.10, 0.15, \dots, 0.90\}$, a piecewise linear quantile regression is fit:

$$Q_\tau(\text{ZCI} \mid \text{PS} = x) = \beta_0 + \beta_1 x + \beta_2 (x - \psi)_+ \quad (23)$$

where $(x - \psi)_+ = \max(0, x - \psi)$ is a hinge function at breakpoint ψ .

Estimation. The parameters $(\beta_0, \beta_1, \beta_2, \psi)$ are estimated jointly by a **profile grid search** (`zci.core.piecewise_qr.profile_search`):

1. Fix a candidate breakpoint ψ_c on a grid spanning the 10th–90th percentile of $\mathbf{s}$.
2. For each ψ_c , build the design matrix $\mathbf{X}(\psi_c) = [\mathbf{1}, \mathbf{s}, (\mathbf{s} - \psi_c)_+]$ and solve the linear quantile regression sub-problem by Linear Programming (LP via `scipy.optimize.linprog`):

$$\min_{\beta, \mathbf{u}, \mathbf{v}} \tau \mathbf{1}^\top \mathbf{u} + (1 - \tau) \mathbf{1}^\top \mathbf{v} \quad \text{s.t.} \quad \mathbf{X}\beta + \mathbf{u} - \mathbf{v} = \mathbf{z}, \quad \mathbf{u}, \mathbf{v} \geq \mathbf{0}. \quad (24)$$

3. Select the ψ_c with the lowest check loss $\rho_\tau(\mathbf{r}) = \sum_i r_i [\tau - 1(r_i < 0)]$.

Wild-bootstrap confidence intervals. `zci.core.piecewise_qr.wild_bootstrap_ci()` follows Feng, He & Hu (2011):

1. Fit the original model; obtain residuals $\hat{r}_i = z_i - \hat{Q}_\tau(s_i)$.
2. For $b = 1, \dots, B$ (default $B = 500$):
 - (a) Draw Mammen’s two-point weights w_i :
$$w_i = \begin{cases} -\frac{\sqrt{5}-1}{2} & \text{w.p. } \frac{\sqrt{5}+1}{2\sqrt{5}}, \\ \frac{\sqrt{5}+1}{2} & \text{w.p. } \frac{\sqrt{5}-1}{2\sqrt{5}}. \end{cases}$$
 - (b) Construct perturbed response: $z_i^* = \hat{Q}_\tau(s_i) + w_i \hat{r}_i$.
 - (c) Re-fit the piecewise QR on (s_i, z_i^*) to obtain $\hat{\beta}^{(b)}, \hat{\psi}^{(b)}$.
3. Confidence intervals: $\alpha/2$ and $1 - \alpha/2$ quantiles of the bootstrap distribution of each parameter (default: 90 % CI).

Note on current status. In the corridor-wide runner (`run_SCDRS_full_pipeline.py`), Stage 5 is currently **commented out** (lines are present but disabled); it is active in the Detroit River runner (`run_DR_full_pipeline.py`). The mathematical description above applies to both.

Artifact written. `05_updated_data.xlsx` (when the stage is enabled).

4 Complementary / Validation Methods

4.1 Stage RDA — Redundancy Analysis

Module. `zci.pipeline.rda_analysis.rda_pipeline()`; core in `zci.core.rda.RDA`.

Purpose. RDA is a **complementary validation** method, not part of the main ZCI construction stream. It quantifies how much of the taxa variation among reference sites is linearly explained by the environmental predictors, and it identifies which environmental variables are most important.

Input.

- Taxa matrix $\mathbf{Y}$ for reference sites ($\mathbf{O}_R$ in octave scale, or Hellinger-transformed).
- Environmental matrix $\mathbf{X}$ ($q = 5$ variables, optionally $\ln(1 + x)$ -transformed and/or z -scored).
- Reference mask from Stage 1 Pollution Score.

Step 1 — Multivariate OLS. Centre both matrices; fit OLS:

$$\hat{\mathbf{Y}} = \mathbf{X}_c \hat{\mathbf{B}}, \quad \hat{\mathbf{B}} = (\mathbf{X}_c^\top \mathbf{X}_c)^{-1} \mathbf{X}_c^\top \mathbf{Y}_c. \quad (25)$$

Step 2 — PCA on fitted values. Eigendecompose the covariance of $\hat{\mathbf{Y}}$:

$$\frac{1}{n-1} \hat{\mathbf{Y}}^\top \hat{\mathbf{Y}} = \mathbf{A} \mathbf{\Lambda} \mathbf{A}^\top, \quad (26)$$

yielding constrained eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots$ and eigenvectors (species scores).

Step 3 — Scores.

- Site scores (scaling 1): $\mathbf{U} = \hat{\mathbf{Y}} \mathbf{A} \mathbf{\Lambda}^{-1/2}$.
- Species scores: $\mathbf{V} = \mathbf{A} \mathbf{\Lambda}^{1/2}$.
- Biplot scores (correlation of $\mathbf{X}_c$ with $\mathbf{U}$): used for environmental-arrow overlays.

Step 4 — Permutation tests. Three types of permutation test are implemented, all based on row-permutation of $\mathbf{Y}_c$ (or residual permutation for term tests):

Test	Null hypothesis
Global	No linear relationship between $\mathbf{Y}$ and $\mathbf{X}$. Pseudo- $F = (\text{Inertia}_{\text{con}}/\text{df}_{\text{model}})/(\text{Inertia}_{\text{res}}/\text{df}_{\text{res}})$.
Per-axis	Each RDA axis individually non-significant. $F_m = \lambda_m/(\text{Inertia}_{\text{res}}/\text{df}_{\text{res}})$.
Per-term (Freedman–Lane)	Each environmental variable individually non-significant, controlling for the others.

Default: 999 permutations per test type.

Outputs. `rda_axes_summary.xlsx`, `rda_terms_summary.xlsx`, `rda_tripplot.png`. No augmented artifact is written — RDA does *not* feed downstream stages.

4.2 Role of ANOVA in the Framework

ANOVA (section 3.3) is **supportive**: it validates that the Ward-clustering-derived habitat groups are ecologically distinct in terms of environmental conditions and taxa composition. It does not feed into the quantitative ZCI construction.

5 Artifact and Intermediate-Value Map

Table 1 lists every stage, its input artifact(s), the new columns added, and the output artifact path.

Table 1: Stage-by-stage artifact chain. “Block” column keys follow the (block, level, variable) MultiIndex convention.

Stage	Input Artifact(s)	New Columns Added	Output Artifact
Stage 1 (PCA)	Original workbook (chemical/raw)	PC1-PC5, Pollution_Score, PC1_sel-PC5_sel (selected PCs)	01_updated_data.xlsx
Stage RDA	Original workbook + Stage 1 artifact	<i>None</i> (tables & figures only)	<i>No artifact; outputs in RDA_analysis/</i>
Stage 2 (Clustering)	Original workbook + Stage 1 artifact	Cluster (reference sites only), Is_Reference	02_updated_data.xlsx
Hindsight	Stage 2 artifact + original workbook	Relabelled Cluster	02_hindsight_updated_data.xlsx
Stage 3 (LDA)	Original workbook + Stage 1 + Hindsight artifacts	Predicted_Cluster, Is_Reference, per-class probabilities, LD1-LD($K-1$)	03_updated_data.xlsx
Stage 4 (NMDS+ZCI)	Original workbook + Stage 1 + Stage 3 artifacts	NMDS1, NMDS2, ZCI, ZCI_Method	04_updated_data.xlsx
Stage 5 (PQR)	Stage 1 + Stage 3 + Stage 4 artifacts	<i>(model parameters; no per-site columns)</i>	05_updated_data.xlsx

6 Quality Assessment and Potential Issues

6.1 ANOVA Assumption Checking

The one-way ANOVA (section 3.3) assumes:

1. **Independence** of observations.
2. **Normality** of the residuals within each group.
3. **Homogeneity of variances** (homoscedasticity) across groups.

Current status. The code in `zci.core.anova.anova_table()` calls `scipy.stats.f_oneway` after an optional pre-transform ("`none`", "`log`", or "`boxcox`"), but **does not** perform any assumption diagnostics:

- No Shapiro–Wilk or Anderson–Darling test for within-group normality.
- No Levene or Brown–Forsythe test for equal variances.
- No Q–Q plot or residual histogram is generated.

Recommended remediation.

1. For each variable $\times$ cluster combination, compute:
 - Shapiro–Wilk W and p -value for within-group normality.
 - Levene’s test (median-based variant, i.e. Brown–Forsythe) for homogeneity of variances.
2. When normality or equal-variance assumptions are violated:
 - Apply a variance-stabilising transform (log, Box–Cox) and re-run; the code already supports `transform="log"` and `transform="boxcox"`.
 - Report Welch’s ANOVA (`scipy.stats.alexandergovern` or the `pingouin` library) as a robust alternative.
 - As a non-parametric backup, report Kruskal–Wallis H with Dunn’s post-hoc tests.
3. In the methods section, explicitly state which transform (if any) was applied, and which variables violated assumptions.
4. Report effect sizes (eta-squared η^2) alongside F and p to assess practical significance.

Impact on results. ANOVA results are used to characterise habitat clusters, not to drive the ZCI computation; thus any bias in p -values affects interpretation tables but not the index itself. Nevertheless, claiming significance for specific variables without verifying assumptions weakens the ecological narrative.

6.2 Bray–Curtis and NMDS: Where Each Operates

Clarification of roles. Bray–Curtis dissimilarity is applied at **two distinct points** in Stage 4:

1. **For NMDS ordination** (section 3.5, Step 3): a full pairwise **D** matrix is computed among all sites within a cluster (plus two synthetic endpoints). This **D** is the input to the iterative non-metric MDS algorithm, which embeds sites in 2-D ordination space.
2. **For BC-Direct ZCI** (section 3.5, Step 7): each site’s relative-abundance vector is compared to the reference and degraded endpoint *centroids* using Bray–Curtis. This computation is independent of NMDS; it operates on the raw **P** matrix directly.

Does the framework correctly reflect contamination-driven community change?

- The NMDS ordination faithfully represents the community *structure* (rank-order of dissimilarities), and its stress value (reported per cluster in `nmds_summary.xlsx`) quantifies the quality of the 2-D approximation.
- The NMDS1 axis is then *artificially oriented* to correlate negatively with pollution ((20)). This is a visualisation convenience but means the NMDS1–Pollution correlation should not be cited as evidence of a contamination gradient.
- The **ZCI itself** (BC-Direct) is computed without any reference to the NMDS coordinates. It directly measures how similar each site’s community is to the reference vs. degraded endpoint in Bray–Curtis space. This is a more defensible measure of contamination-induced community shift because it avoids the information loss inherent in projecting to 2-D.
- The ZCI–Pollution Score correlation (Pearson and Spearman) is computed and reported; a high positive correlation supports the interpretation that the index captures contamination-driven community change.

Potential issues.

- **Endpoint sensitivity:** the n_{ep} parameter controls how many extreme sites define the reference/degraded centroids. Small n_{ep} (e.g. 3 for Cluster 2) makes the centroid sensitive to individual sites.
- **NMDS stress:** if stress exceeds ~ 0.20 the 2-D representation is unreliable; report and evaluate per cluster.
- **Cluster-specific ZCI:** the ZCI is computed separately within each habitat cluster, so cross-cluster comparisons require caution.

6.3 LDA Cross-Validation Leakage

As noted in section 3.4, the Monte Carlo CV in `zci.core.lda.monte_carlo_cv()` applies `StandardScaler.fit_transform` to the *entire* reference-site environmental matrix before splitting into CV folds. This means every training fold has indirect information about the test-fold means and variances. The resulting CV accuracy may be **slightly optimistic**.

Recommended fix. Move the standardisation inside each CV split: fit the scaler on the training fold only, then transform the test fold using the training scaler.

7 TITAN Embedding Plan

7.1 Background

TITAN (Threshold Indicator Taxa ANalysis; Baker & King, 2010) detects taxon-specific change points along a continuous environmental gradient. It complements the current pipeline by answering a question the existing stages do not:

At what pollution level does each individual taxon show the sharpest decline (or increase), and what is the aggregate community-level threshold?

Existing notebook: `notebooks/10.TITAN.ipynb` contains a simulated-data demonstration and a real-data application (Part 2) using the corridor-wide pollution-score gradient.

7.2 TITAN Algorithm Summary

Given an ordered gradient $x_1 \leq x_2 \leq \dots \leq x_n$ (Pollution Score) and a taxa-by-site abundance matrix $\mathbf{Y}$:

1. **Candidate partitions.** For each candidate split value x_c (excluding the 5 smallest and 5 largest values to avoid edge effects), divide sites into two groups: $\{i : x_i \leq x_c\}$ and $\{i : x_i > x_c\}$.
2. **Indicator Value.** For each taxon j at each split x_c , compute the Indicator Value (IndVal; Dufrêne & Legendre, 1997):

$$\text{IndVal}_j = A_j \times B_j \times 100, \quad (27)$$

where A_j (specificity) is the mean abundance in the target group relative to the overall mean, and B_j (fidelity) is the proportion of sites in the target group where taxon j occurs.

3. **z -score.** Compare $\text{IndVal}_j(x_c)$ to a null distribution obtained by permuting site labels (250–1000 permutations), yielding $z_j(x_c)$.
4. **Change-point selection.** The change point for taxon j is $x_c^{*(j)} = \arg \max_{x_c} z_j(x_c)$.
5. **Direction assignment.**
 - z^- (sensitive / clean-water indicator): taxon abundance peaks in the low- x group.
 - z^+ (tolerant / pollution indicator): taxon abundance peaks in the high- x group.
6. **Bootstrap validation** (B_{boot} resamples, e.g. 500):
 - **Purity** ≥ 0.95 : $\geq 95\%$ of bootstrap replicates assign the taxon to the same direction.
 - **Reliability** ≥ 0.95 : $\geq 95\%$ yield a significant response ($p \leq 0.05$).

Only taxa passing both filters are retained.

7. **Community-level thresholds.** Sum the filtered z -scores across all reliable taxa in each direction:

$$\text{sum}(z^-)(x) = \sum_{j \in \text{declining}} z_j^-(x), \quad \text{sum}(z^+)(x) = \sum_{j \in \text{increasing}} z_j^+(x). \quad (28)$$

The x -value where each sum peaks defines the community-level change point.

7.3 Integration Points in the Current Framework

TITAN should be run **per habitat cluster** (using Stage 3 Predicted_Cluster labels) with the following inputs:

TITAN input	Source	Code path
Gradient x	Pollution.Score	Stage 1 artifact
Taxa matrix $\mathbf{Y}$	Relative-abundance $\mathbf{P}$	<code>octave_to_relative_abundance(taxa_all)</code>
Cluster grouping	Predicted_Cluster	Stage 3 artifact

Execution. TITAN2 is an R package; the existing notebook (`notebooks/10_TITAN.ipynb`) calls it via `rpy2` magic cells. A production integration would:

1. Load Stage 1, Stage 3, and Stage 4 artifacts plus the original taxa block.
2. For each cluster, filter sites and pass the Pollution Score vector and taxa matrix to `TITAN2::titan()` in R.
3. Retrieve `sppmax` (taxon-level change points, purity, reliability) and `sumz` (community-level $\text{sum}(z)$ curves) back into Python.
4. Overlay TITAN thresholds on the ZCI vs. Pollution Score scatter plot:
 - Vertical lines at the Pollution Score community thresholds ($\text{sum}(z^-)$ peak, $\text{sum}(z^+)$ peak).
 - Horizontal lines at optionally detected ZCI thresholds if TITAN is also run with ZCI as a gradient.
 - Shaded bands for bootstrap confidence intervals.

Theoretical rationale. TITAN adds a **taxon-resolution layer** that the current framework lacks. While the ZCI gives a single aggregate condition score per site, TITAN identifies *which* taxa are responsible for community shifts and at *what gradient value* they change. Specifically:

- It answers whether the **reference-site threshold** used to define $\mathcal{R}$ (bottom 20 %) corresponds to an ecologically meaningful change point, or is arbitrary.
- It reveals whether different taxa respond at different pollution levels (gradual turnover) or at a single critical threshold (abrupt regime shift).
- The community $\text{sum}(z)$ peaks provide an independent estimate of the “reference/degraded boundary” that can be compared with the Stage 5 piecewise-QR breakpoint.

Caveats.

- TITAN was designed for site $\times$ gradient threshold detection; using ZCI as a secondary gradient is a non-standard but creative extension.
- Results depend on the cluster assignment from Stage 3.
- The 5th-to-95th percentile trimming used by TITAN2 means extreme-gradient thresholds are not detectable.
- Bootstrap resampling (≥ 500 replicates) is recommended for publication; the notebook prototype used 100 for speed.

8 Summary of Recommendations

1. **ANOVA assumptions:** add Shapiro–Wilk and Levene’s tests; report Welch or Kruskal–Wallis as robust alternatives where assumptions fail.
2. **LDA CV leakage:** move `StandardScaler` fitting inside each CV fold.
3. **NMDS axis-flip caveat:** state explicitly in the thesis that NMDS1 orientation is enforced, not discovered.
4. **Endpoint sensitivity:** report ZCI stability under varying n_{ep} .
5. **PCA variance scaling:** note explicitly that PCA is performed on log-transformed but not unit-scaled variables, so higher-variance contaminants dominate.
6. **TITAN integration:** embed per-cluster TITAN into the framework after Stage 3, using the Pollution Score gradient and relative-abundance taxa matrix; overlay community thresholds on ZCI–Pollution plots.

References

- [1] Baker, M.E. & King, R.S. (2010). A new method for detecting and interpreting biodiversity and ecological community thresholds. *Methods in Ecology and Evolution*, 1, 25–37.
- [2] Dufrêne, M. & Legendre, P. (1997). Species assemblages and indicator species: the need for a flexible asymmetrical approach. *Ecological Monographs*, 67, 345–366.
- [3] Feng, X., He, X. & Hu, J. (2011). Wild bootstrap for quantile regression. *Biometrika*, 98, 995–999.